

CS 171 / EE 142: Intro to ML and DM

Christian Shelton

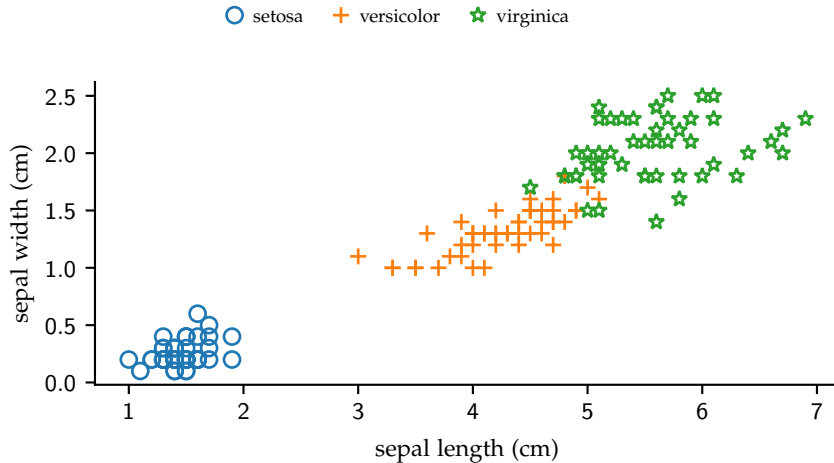
UC Riverside

Slide Set 11: Nearest Neighbor

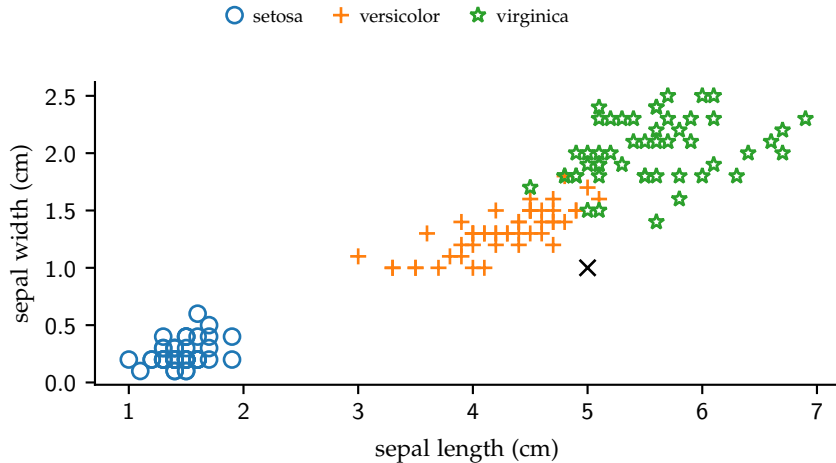


- From UC Riverside
 - ▶ CS 171 / EE 142: Introduction to Machine Learning and Data Mining
 - ▶ Professor Christian Shelton
- DO NOT REDISTRIBUTE
 - ▶ For use only by enrolled students in the course

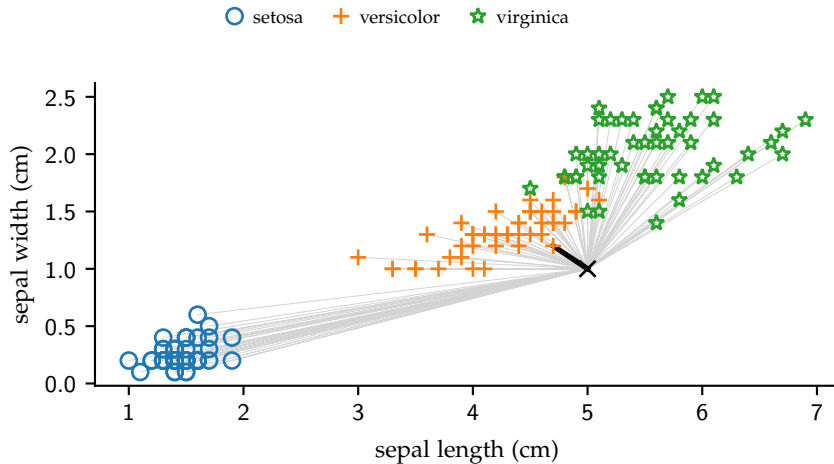
k -Nearest Neighbor



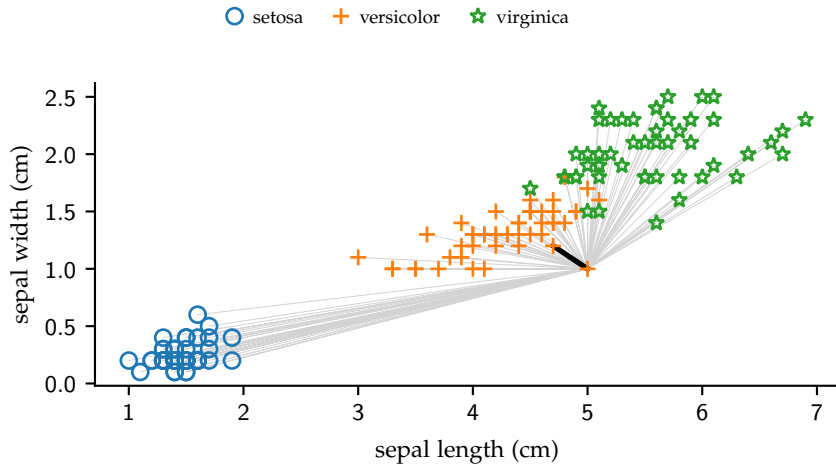
k-Nearest Neighbor



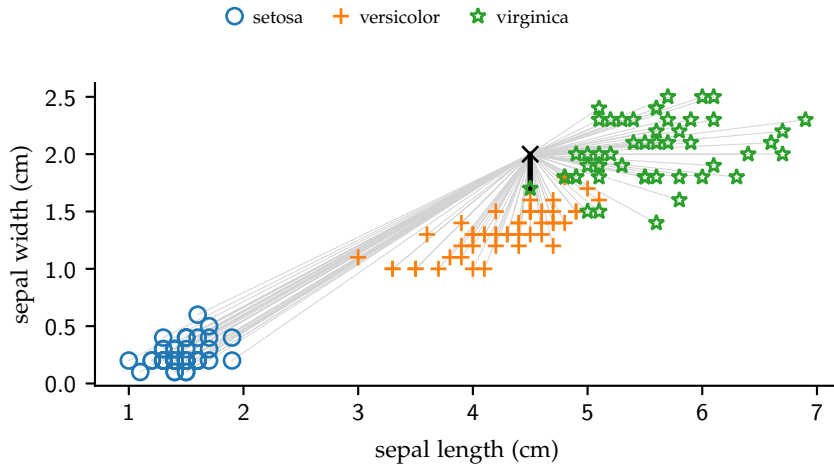
k -Nearest Neighbor



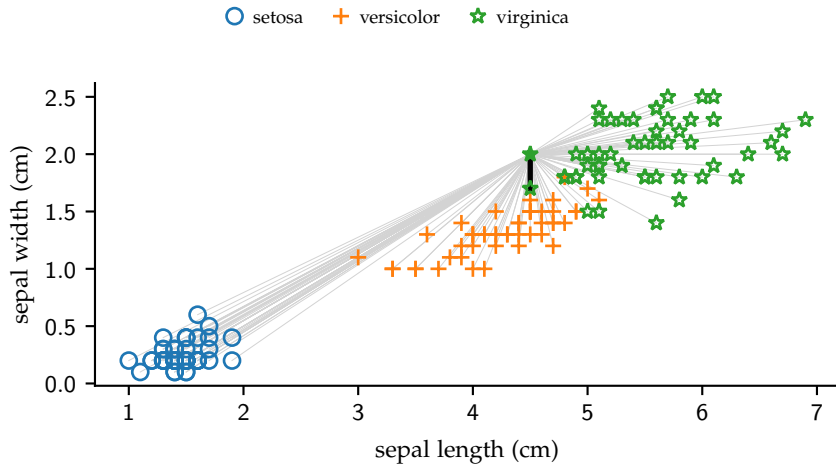
k -Nearest Neighbor



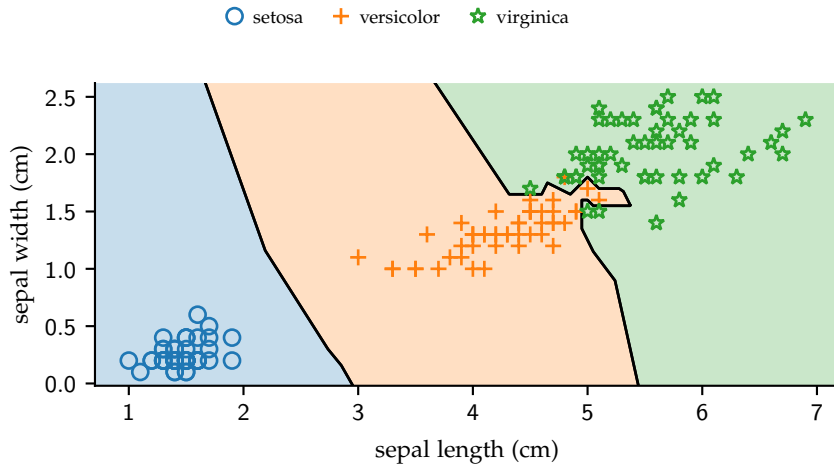
k -Nearest Neighbor



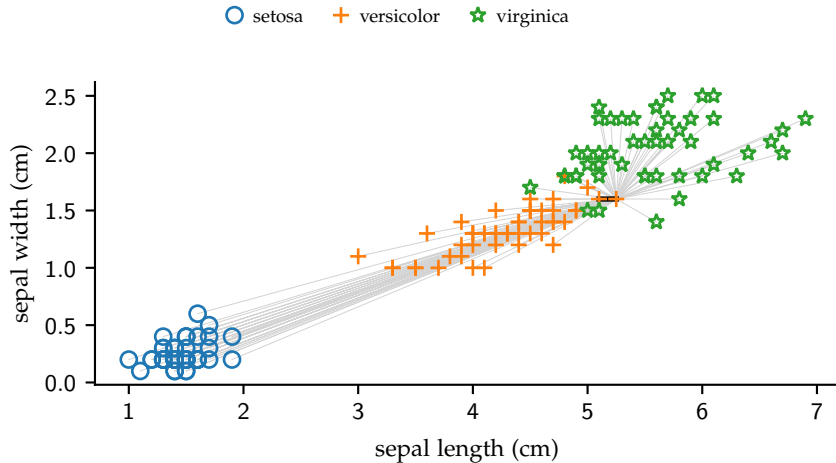
k -Nearest Neighbor



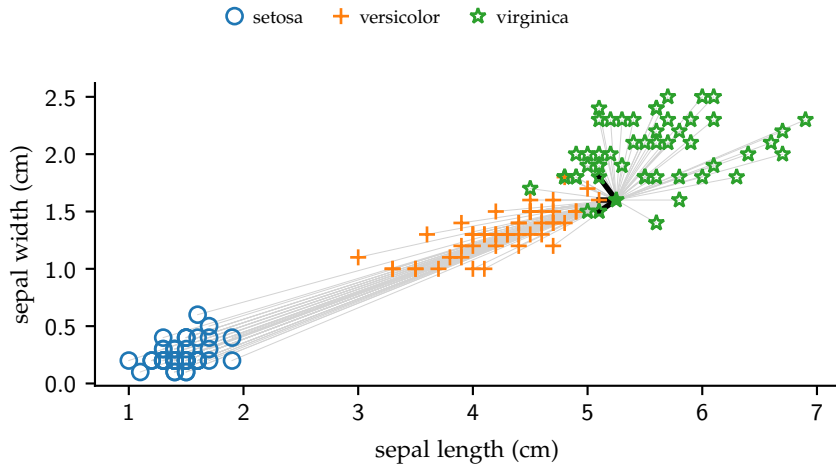
k -Nearest Neighbor



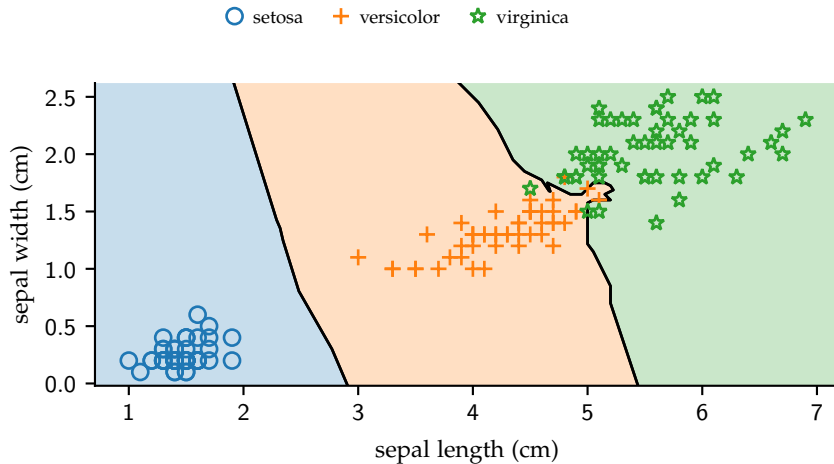
k -Nearest Neighbor



k -Nearest Neighbor



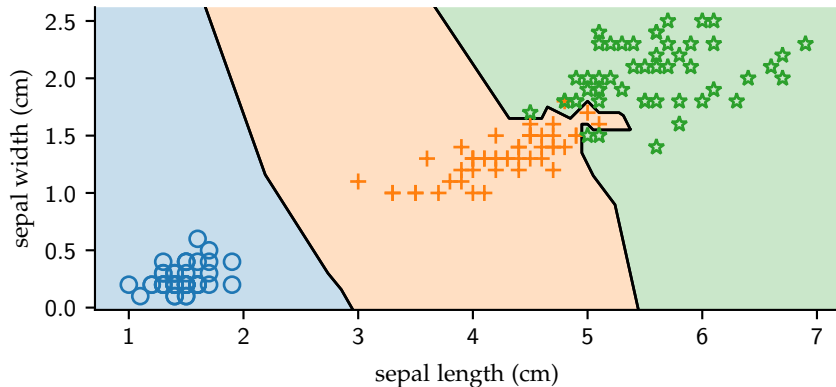
k -Nearest Neighbor



k -Nearest Neighbor, k comparison

$k = 1$

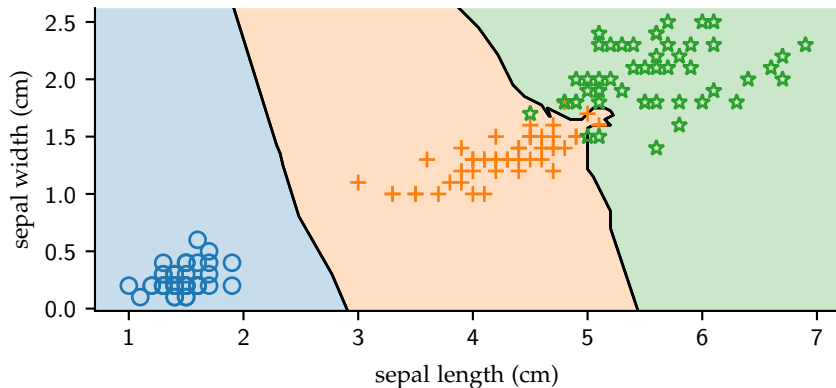
○ setosa + versicolor ☆ virginica



k -Nearest Neighbor, k comparison

$k = 3$

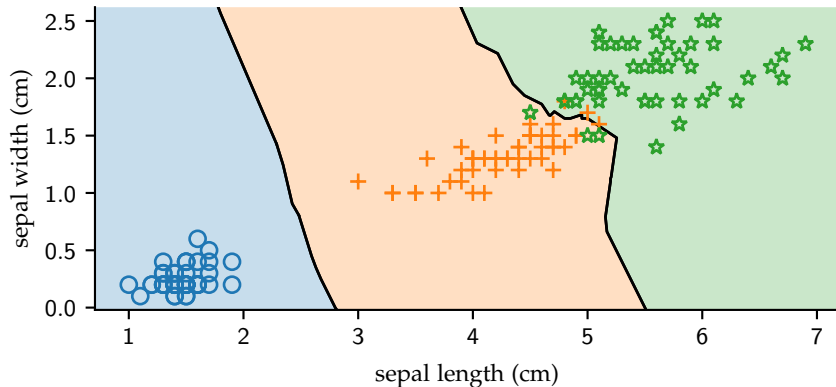
○ setosa + versicolor ☆ virginica



k -Nearest Neighbor, k comparison

$k = 5$

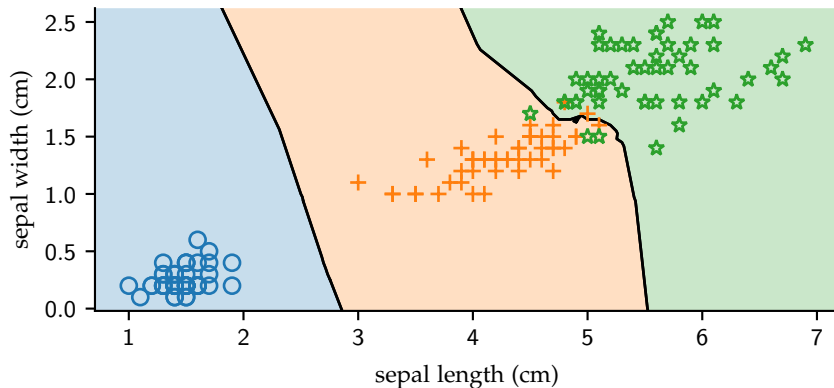
○ setosa + versicolor ☆ virginica



k -Nearest Neighbor, k comparison

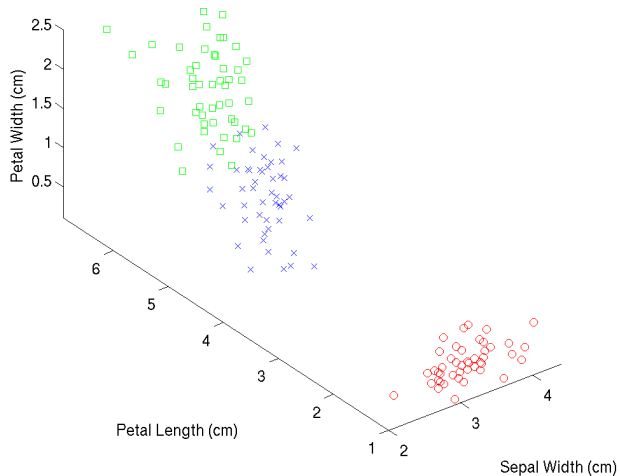
$k = 7$

○ setosa + versicolor ☆ virginica



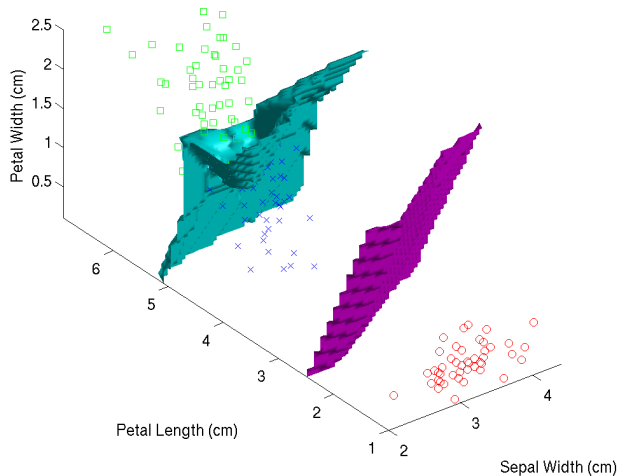
k -Nearest Neighbor, 3D

$k = 1$



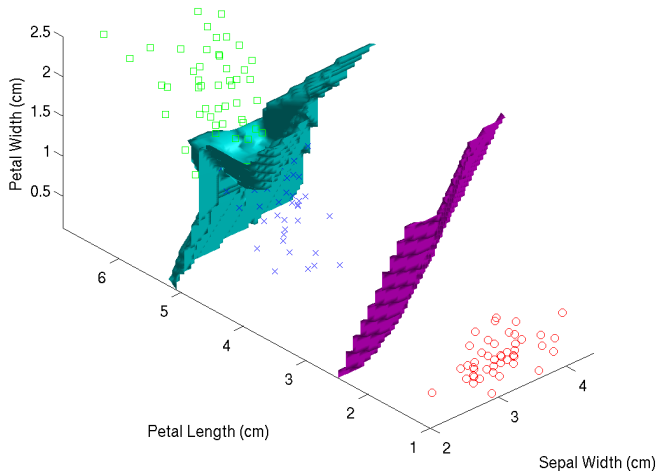
k -Nearest Neighbor, 3D

$k = 1$



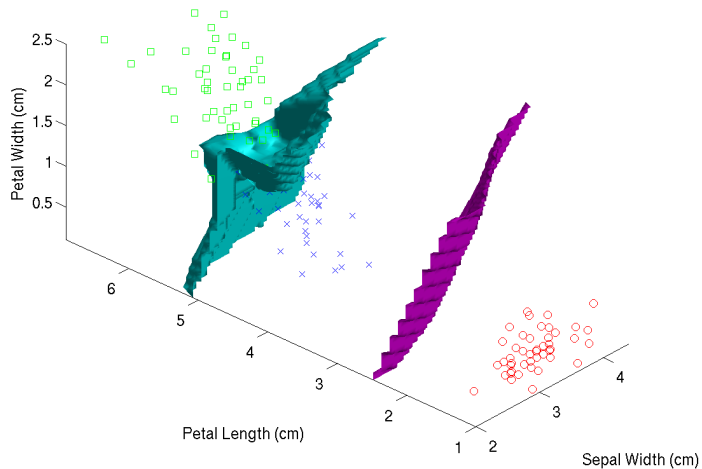
k -Nearest Neighbor, 3D

$k = 1$



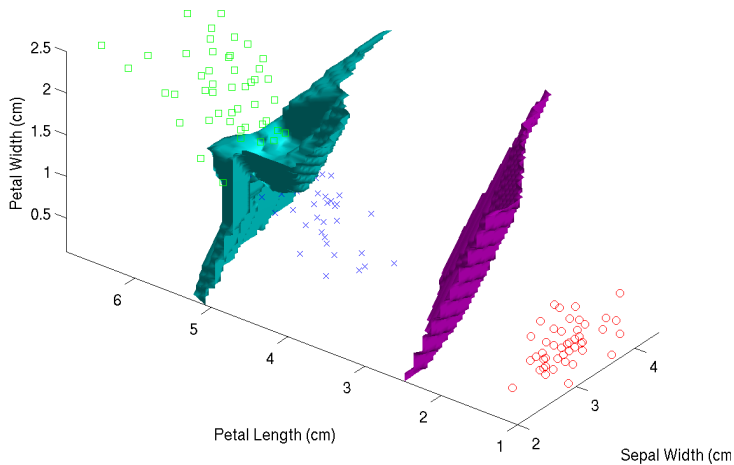
k -Nearest Neighbor, 3D

$k = 1$



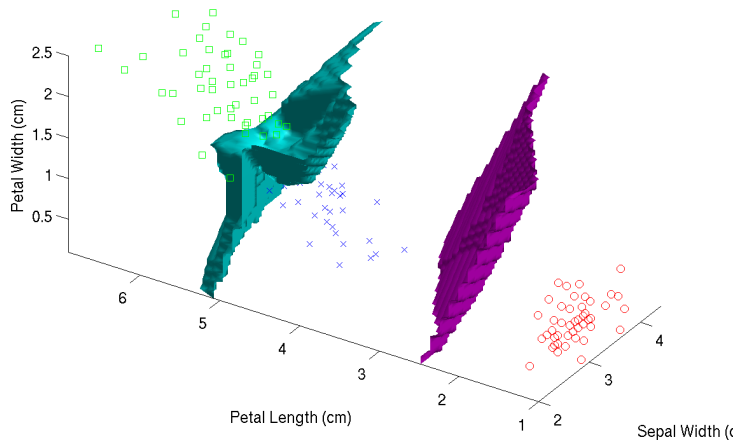
k -Nearest Neighbor, 3D

$k = 1$



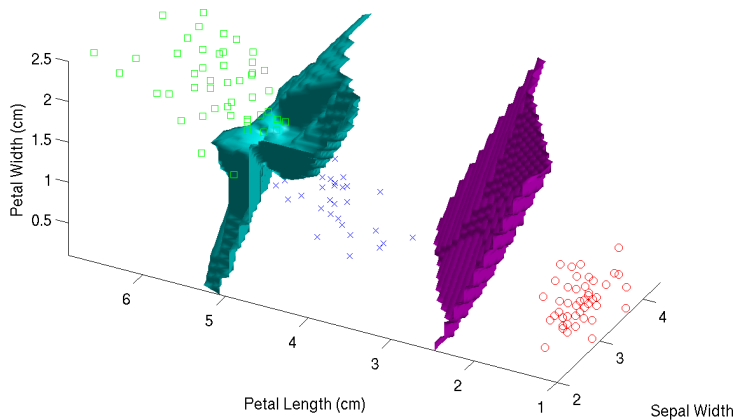
k -Nearest Neighbor, 3D

$k = 1$



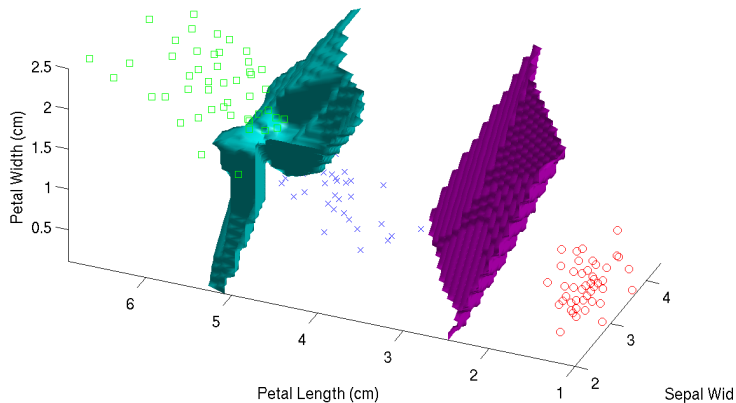
k -Nearest Neighbor, 3D

$k = 1$



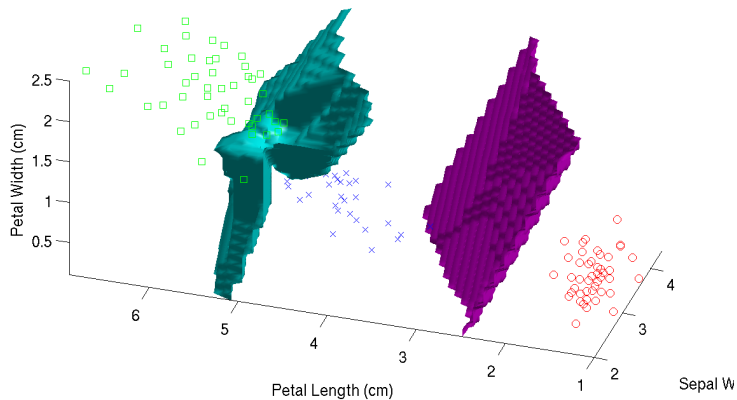
k -Nearest Neighbor, 3D

$k = 1$



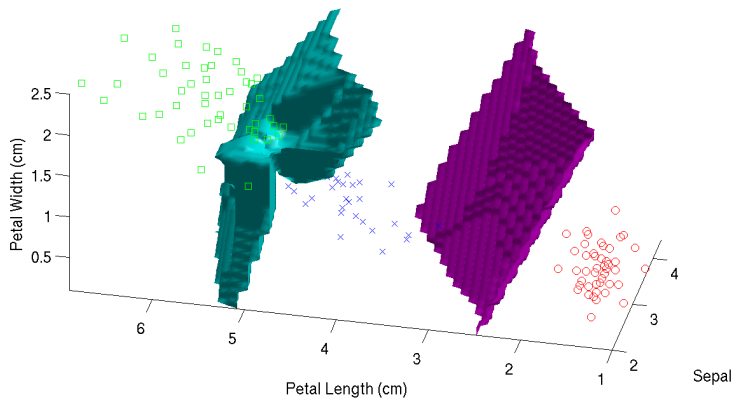
k -Nearest Neighbor, 3D

$k = 1$



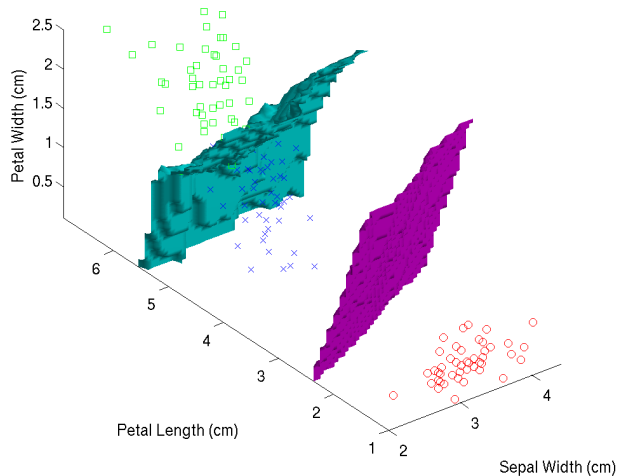
k -Nearest Neighbor, 3D

$k = 1$



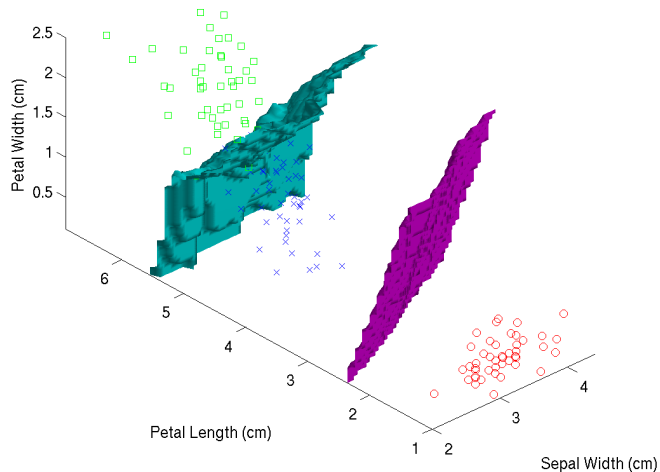
k -Nearest Neighbor, 3D

$k = 5$



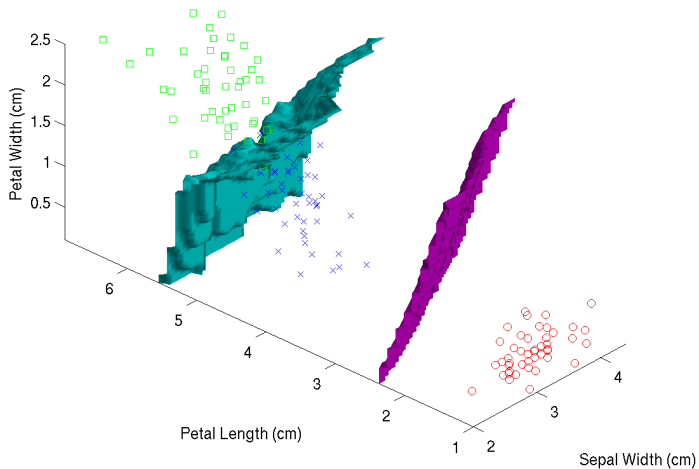
k -Nearest Neighbor, 3D

$k = 5$



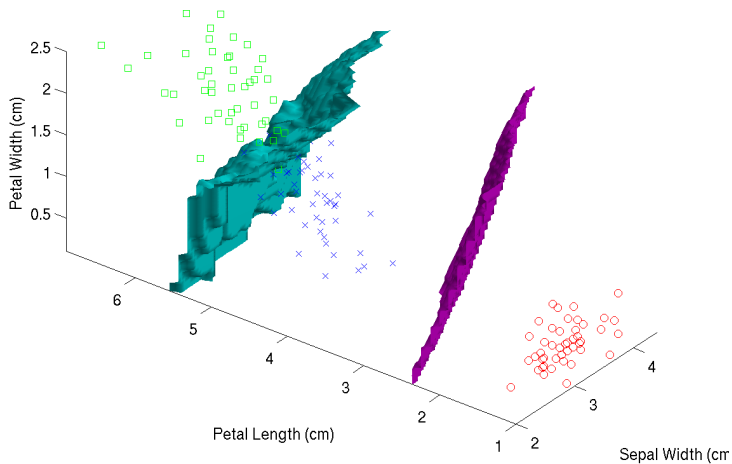
k -Nearest Neighbor, 3D

$k = 5$



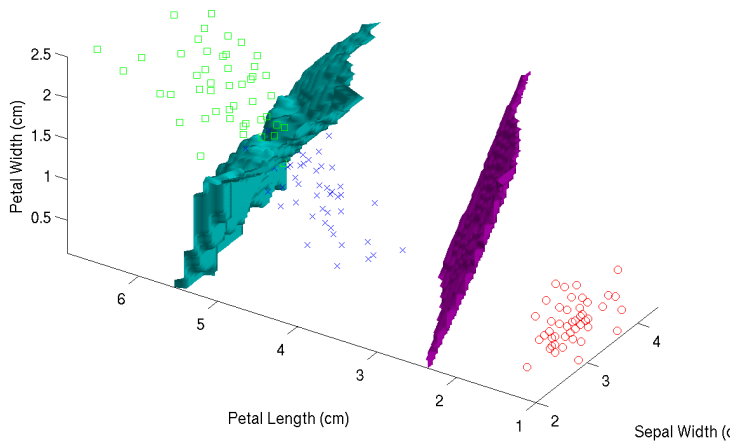
k -Nearest Neighbor, 3D

$k = 5$



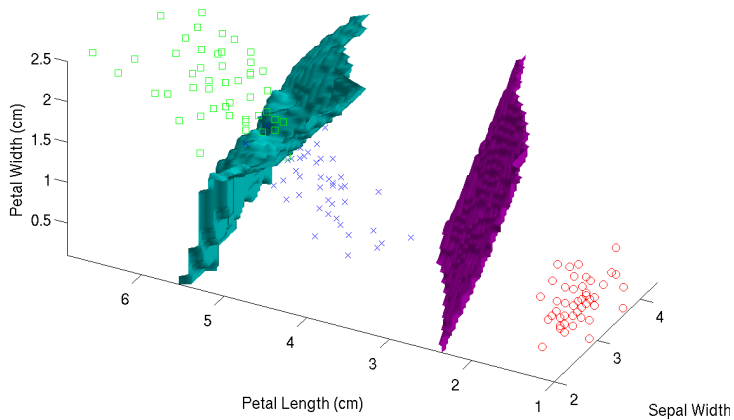
k -Nearest Neighbor, 3D

$k = 5$



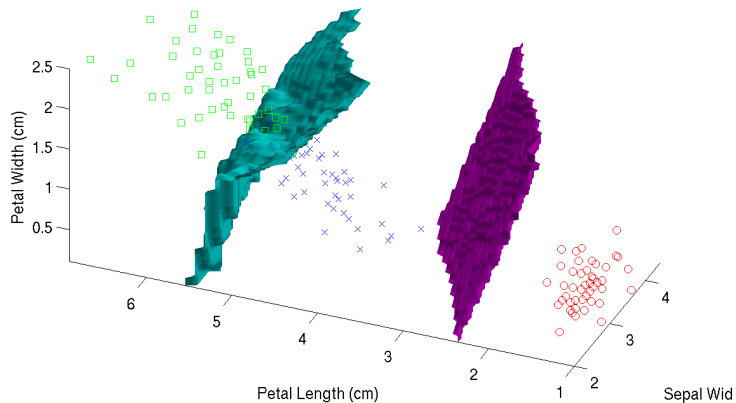
k -Nearest Neighbor, 3D

$k = 5$



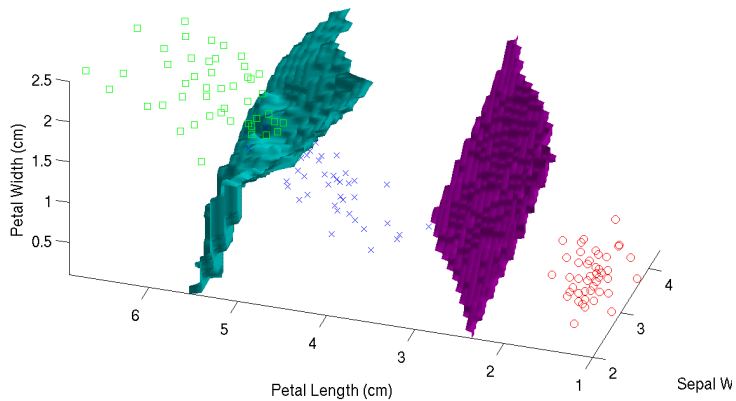
k -Nearest Neighbor, 3D

$k = 5$



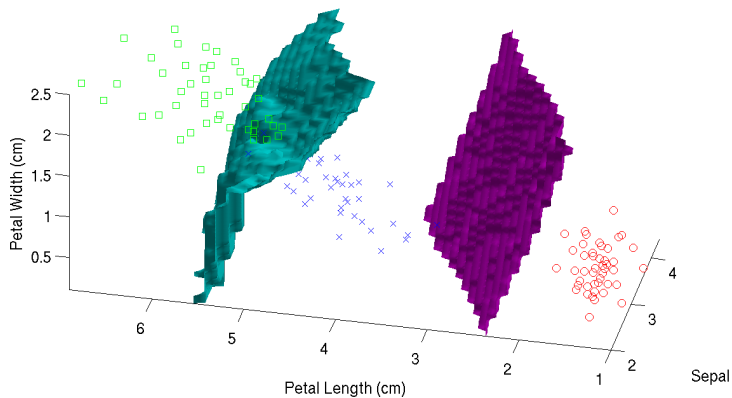
k -Nearest Neighbor, 3D

$k = 5$



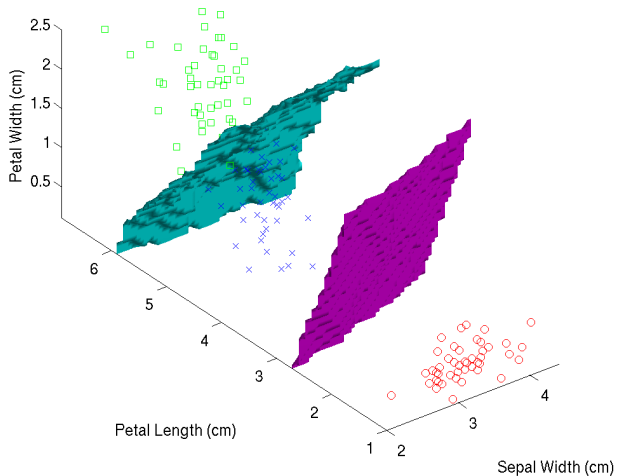
k -Nearest Neighbor, 3D

$k = 5$



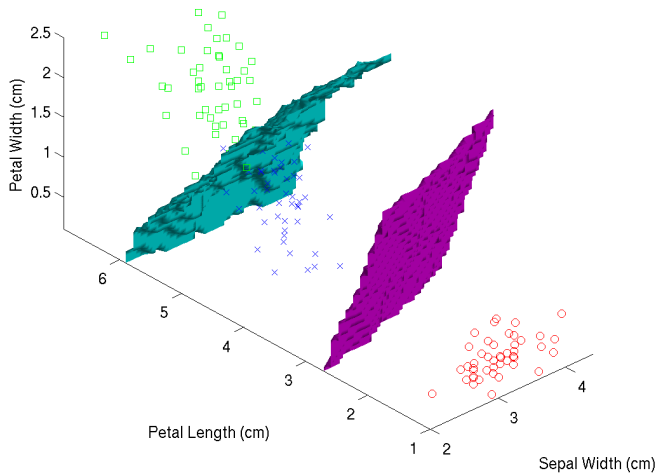
k -Nearest Neighbor, 3D

$k = 15$



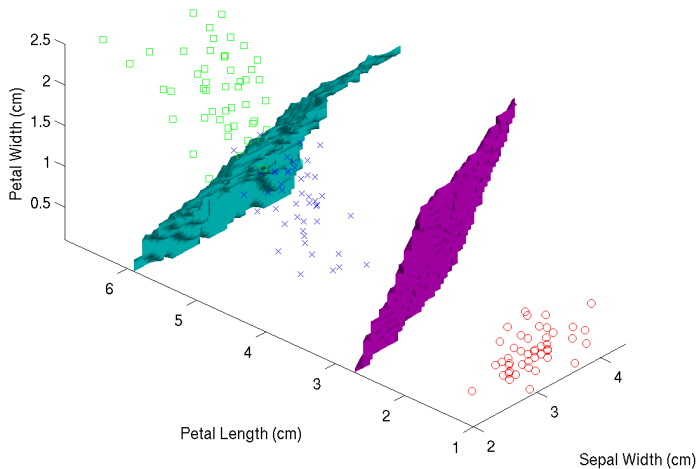
k -Nearest Neighbor, 3D

$k = 15$



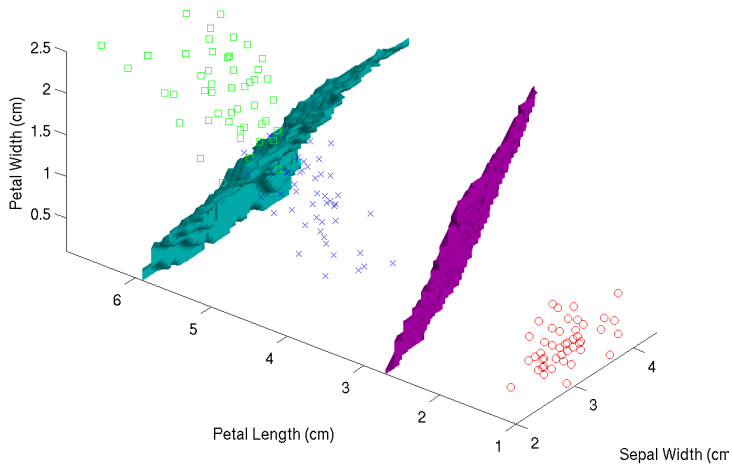
k -Nearest Neighbor, 3D

$k = 15$



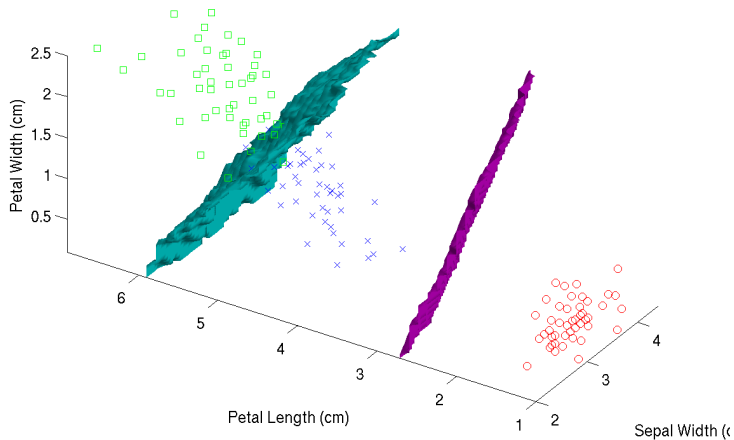
k -Nearest Neighbor, 3D

$k = 15$



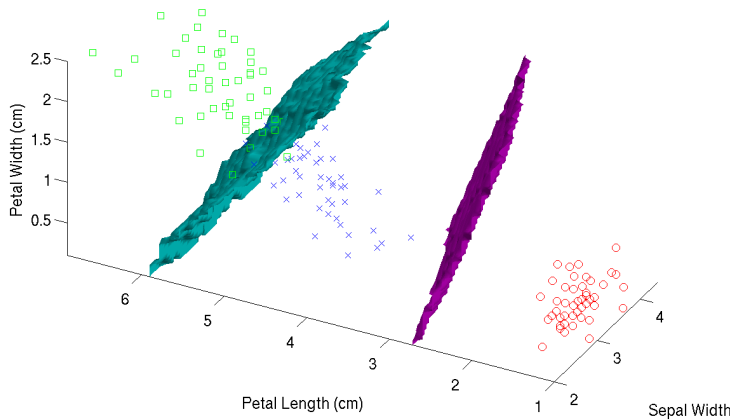
k -Nearest Neighbor, 3D

$k = 15$



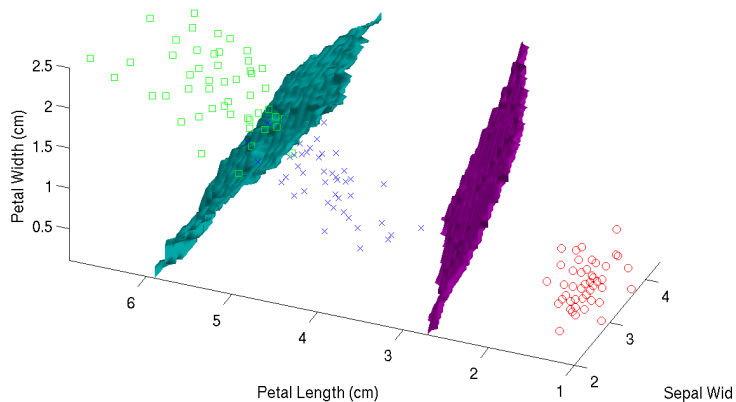
k -Nearest Neighbor, 3D

$k = 15$



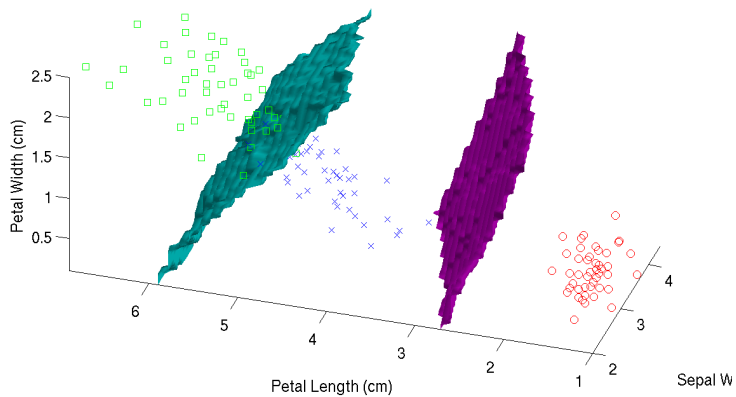
k -Nearest Neighbor, 3D

$k = 15$



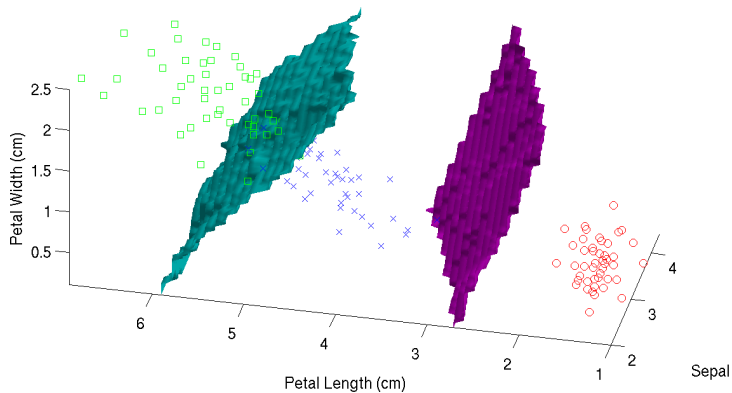
k -Nearest Neighbor, 3D

$k = 15$



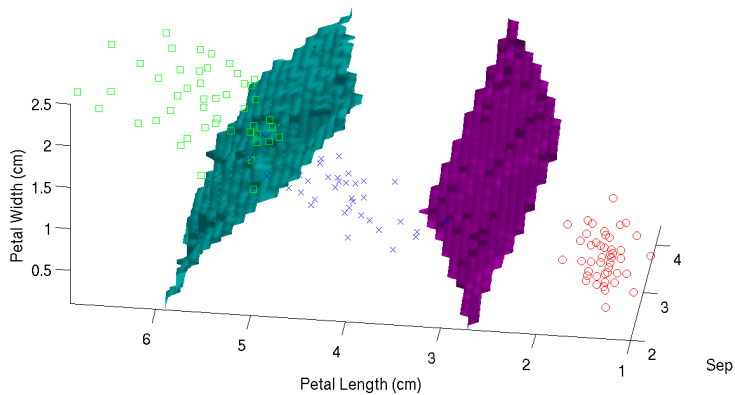
k -Nearest Neighbor, 3D

$k = 15$



k -Nearest Neighbor, 3D

$k = 15$



k -Nearest Neighbor Notes

- Point a and point b have (Euclidean) distance

$$d(\vec{a}, \vec{b}) = \sqrt{\sum_i (a_i - b_i)^2}$$

- A lazy method: no work done at training time, all work done at testing time

1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

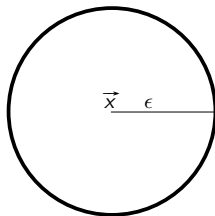
$$\text{error}_{\text{opt}}(\vec{x}) = \min_y (1 - P(y \mid \vec{x}))$$

1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

$$\text{error}_{\text{opt}}(\vec{x}) = \min_y (1 - P(y \mid \vec{x}))$$

How well will 1-NN do as $m \rightarrow \infty$?

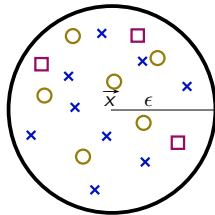


1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

$$\text{error}_{\text{opt}}(\vec{x}) = \min_y (1 - P(y \mid \vec{x}))$$

How well will 1-NN do as $m \rightarrow \infty$?

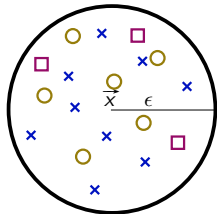


1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

$$\text{error}_{\text{opt}}(\vec{x}) = \min_y (1 - P(y \mid \vec{x}))$$

How well will 1-NN do as $m \rightarrow \infty$?



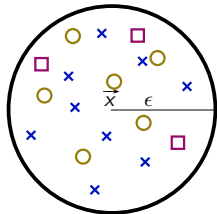
$$\text{error}_{1\text{-NN}}(x) = \sum_y P(y \mid \vec{x}) (1 - P(y \mid \vec{x}))$$

1-NN versus Bayes Optimal

If we knew $P(y \mid \vec{x})$ we could produce the Bayes-optimal classifier.
How good would it be?

$$\text{error}_{\text{opt}}(\vec{x}) = \min_y (1 - P(y \mid \vec{x}))$$

How well will 1-NN do as $m \rightarrow \infty$?



$$\text{error}_{1\text{-NN}}(\vec{x}) = \sum_y P(y \mid \vec{x}) (1 - P(y \mid \vec{x}))$$

$$\text{error}_{\text{opt}}(\vec{x}) \leq \text{error}_{1\text{-NN}}(\vec{x}) \leq 2\text{error}_{\text{opt}}(\vec{x})$$

k -NN versus Bayes Optimal

What about k -NN?

k -NN versus Bayes Optimal

What about k -NN?

k must be a function of m (number of examples) to get consistency.

If

- $\lim_{m \rightarrow \infty} k(m) = \infty$, and

- $\lim_{m \rightarrow \infty} k(m)/m = 0$

then, k -NN converges to the Bayes-optimal error rate

k -NN versus Bayes Optimal

What about k -NN?

k must be a function of m (number of examples) to get consistency.

If

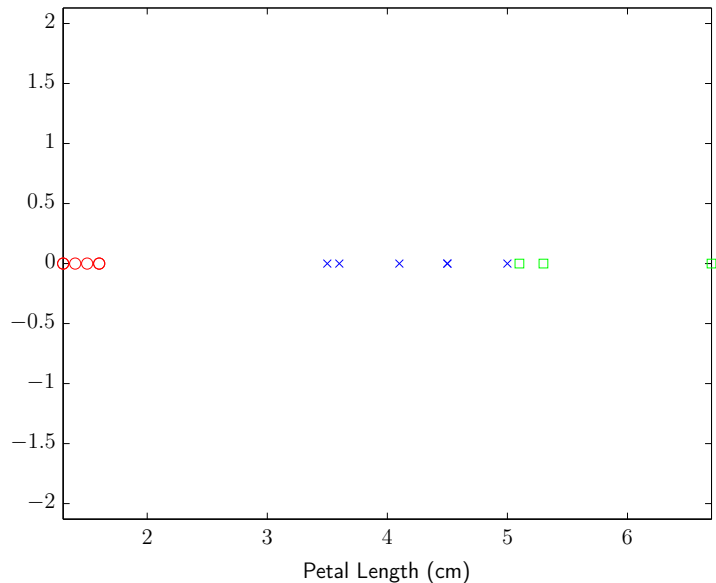
- $\lim_{m \rightarrow \infty} k(m) = \infty$, and

- $\lim_{m \rightarrow \infty} k(m)/m = 0$

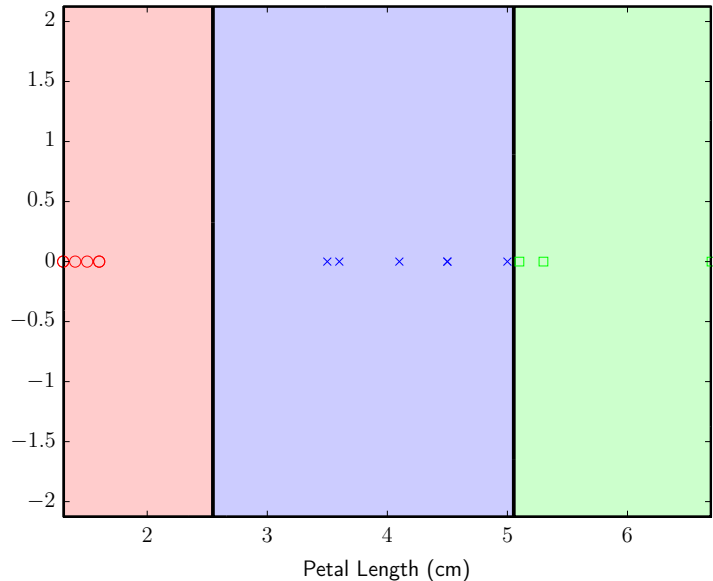
then, k -NN converges to the Bayes-optimal error rate

But if $m < \infty$, almost nothing is known.

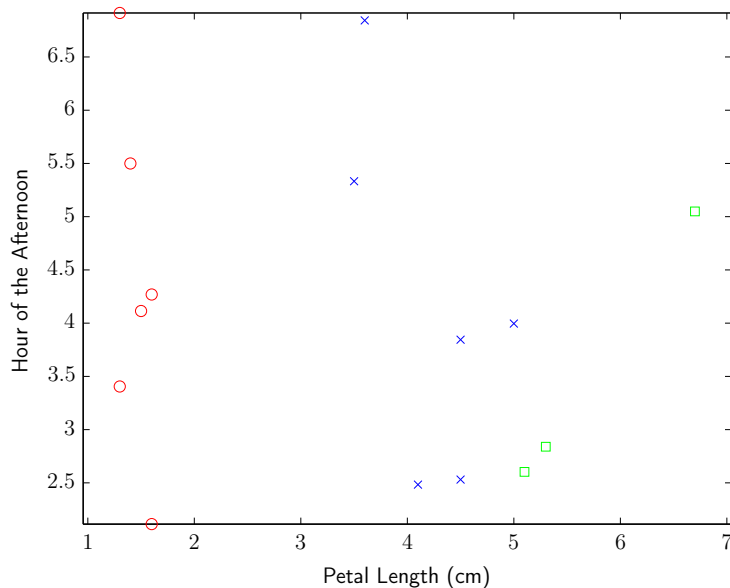
Irrelevant features



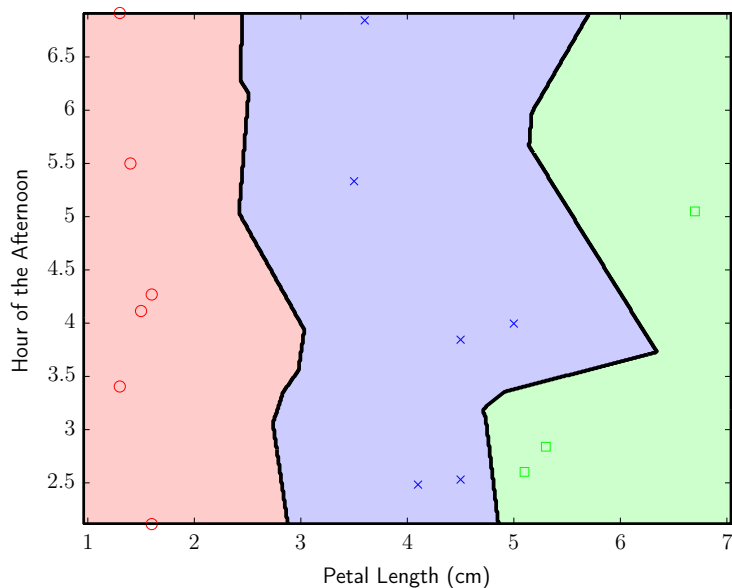
Irrelevant features



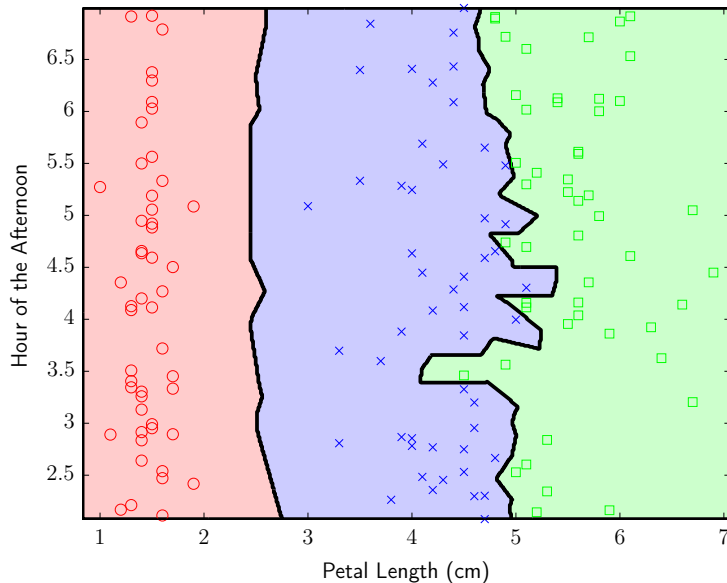
Irrelevant features



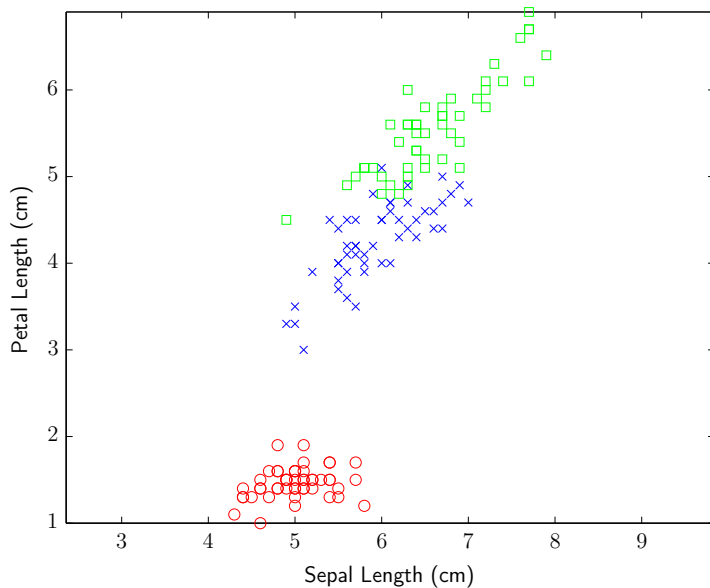
Irrelevant features



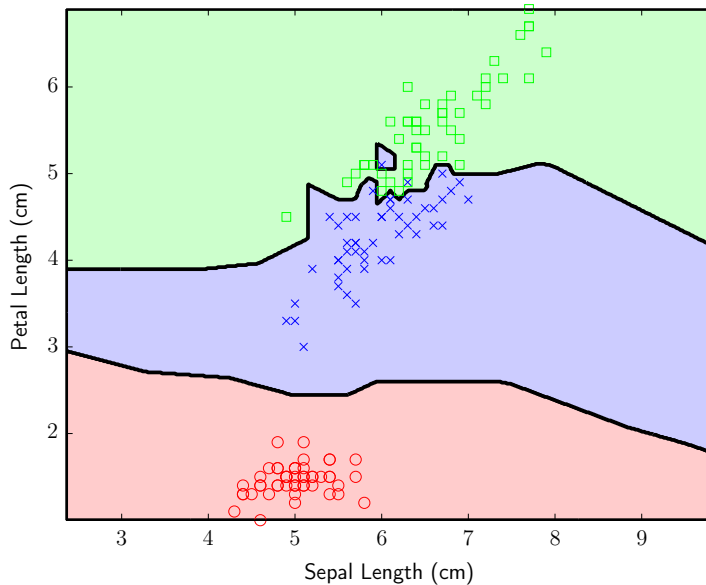
Irrelevant features



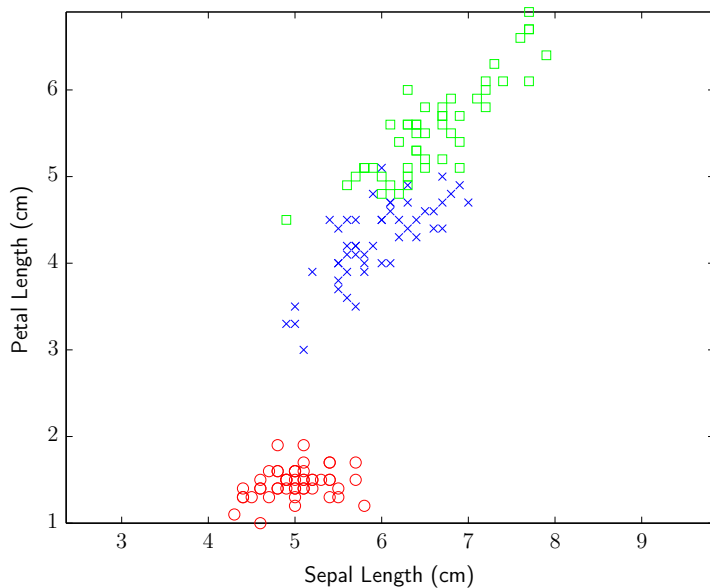
Feature scaling



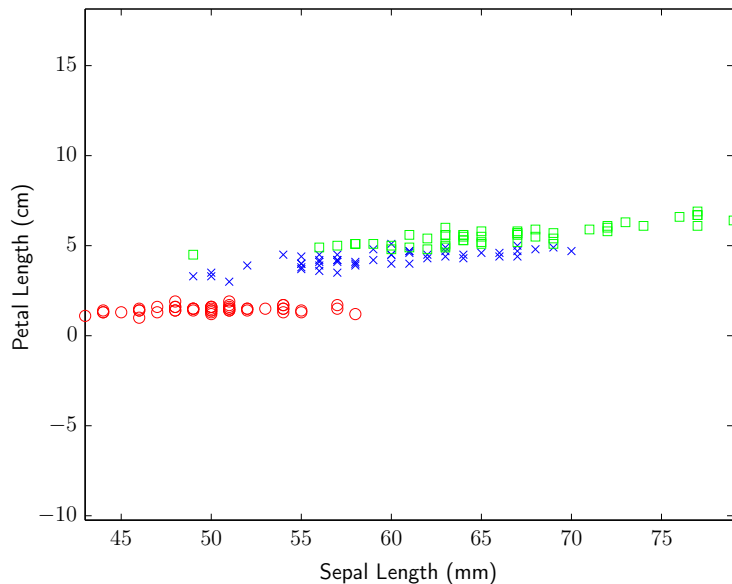
Feature scaling



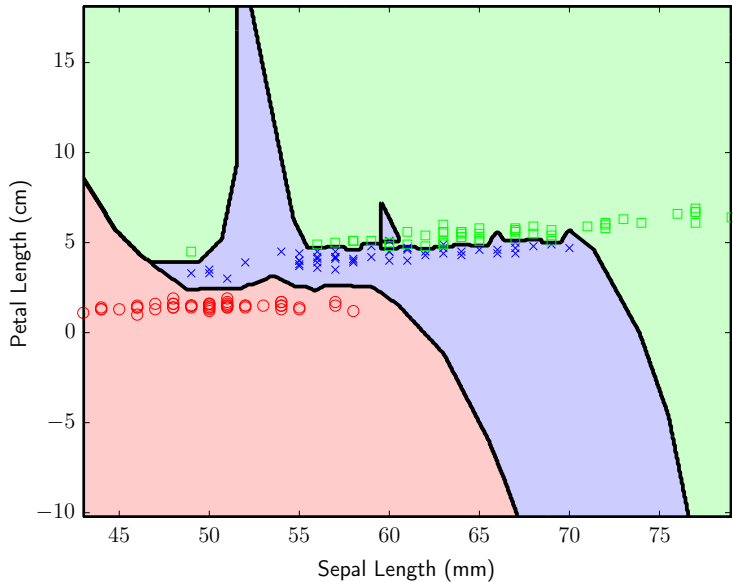
Feature scaling



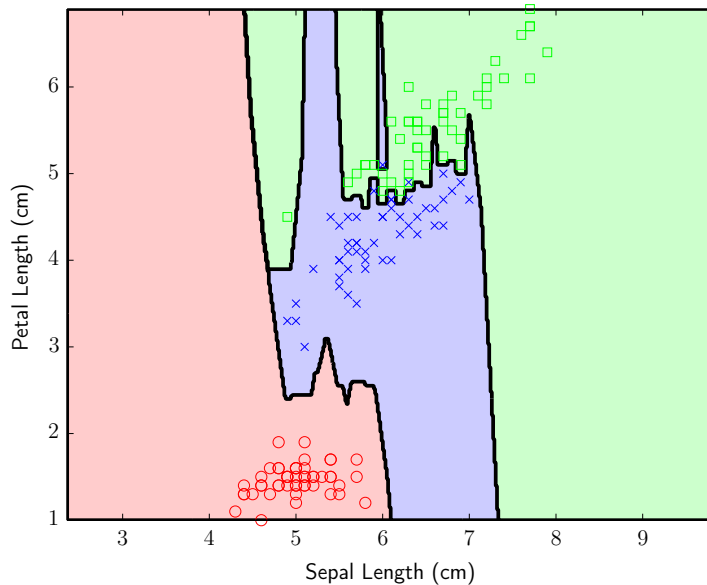
Feature scaling



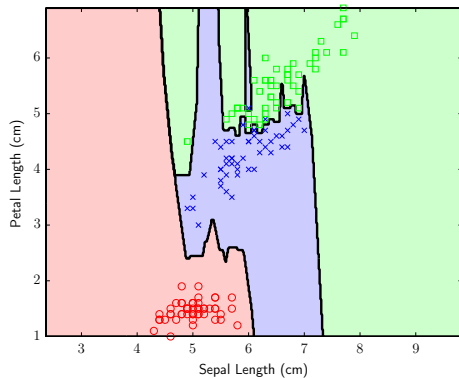
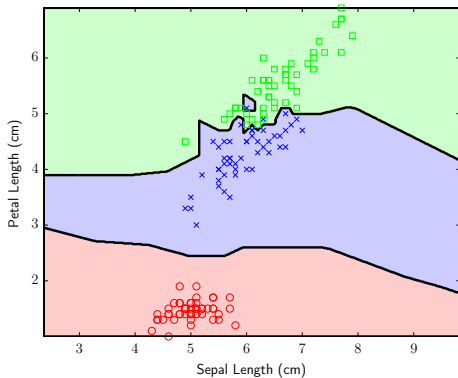
Feature scaling



Feature scaling



Feature scaling



Feature scaling

- Not as simple as “pick same units for all features”
 - ▶ What about temperature and length?
 - ▶ Is petal length really the same as sepal length?
- What about discrete features?
 - ▶ Need distance between them
 - ▶ Binary can be 0 if same, 1 if different, but then should it be scaled?
 - ▶ Non-binary may be ordinal or categorical
- Irrelevant features are just an extreme example (scaling should be 0!)

Feature scaling

Given two points, a training point $\vec{x} = [x_1, x_2]^T$ and a testing point $\vec{z} = [z_1, z_2]^T$,
2D Euclidean distance:

$$d(\vec{x}, \vec{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}$$

Feature scaling

Given two points, a training point $\vec{x} = [x_1, x_2]^T$ and a testing point $\vec{z} = [z_1, z_2]^T$,
2D Euclidean distance:

$$d(\vec{x}, \vec{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}$$

If we scale the first feature by 10 (say measure in mm instead of cm):
2D Euclidean distance:

$$d(\vec{x}, \vec{z}) = \sqrt{(10x_1 - 10z_1)^2 + (x_2 - z_2)^2} = \sqrt{100(x_1 - z_1)^2 + (x_2 - z_2)^2}$$

which scales the importance of similarity (or “exaggerates” the dissimilarity) of feature 1.

Distance Metrics

The Euclidean distance isn't the only method to measure the dissimilarity of two points. Here are some common distance metrics:

- Euclidean (also known as the L_2 metric): ¹

$$d(\vec{x}, \vec{z}) = \left(\sum_{i=1}^n (x_i - z_i)^2 \right)^{1/2}$$

- Manhattan (also known as the L_1 metric):

$$d(\vec{x}, \vec{z}) = \left(\sum_{i=1}^n |x_i - z_i| \right)$$

- String edit distance: Given two strings (not vectors), the minimum number of edits (insert symbol, delete symbol, change symbol) necessary to change one string into the other.
- Graph edit distance: similar to strings, but with changes to measure the distance between two graphs

¹The square root is not necessary if we only need to calculate which is closer; using the squared distance is equivalent for this purpose.

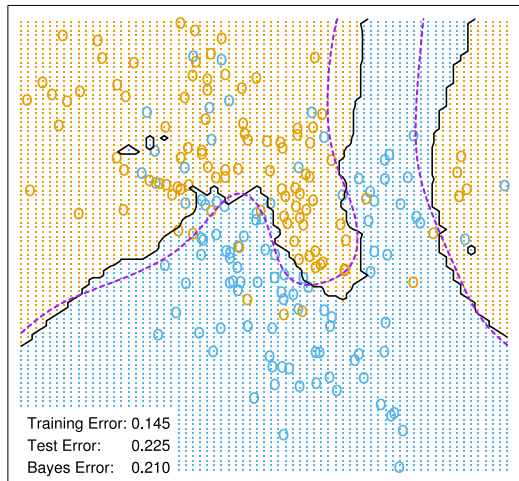
Which metric/scaling

- Even if you don't explicitly pick a scaling or metric, you are implicitly picking one.
- Irrelevant attributes is an extreme case of needing to scale the attribute (by 0)
- Euclidean distance is rotational invariant, but often this is not necessary.
- Selection is a way of injecting your own knowledge into the problem to help the learning.
 - ▶ Best metric is one that already “solves” the problem and gives a distance of 0 to members of the same class.
- k -NN (with k properly chosen) will converge to the optimal solution with ∞ data regardless.
- However, with finite data (the common case!) metric matters.

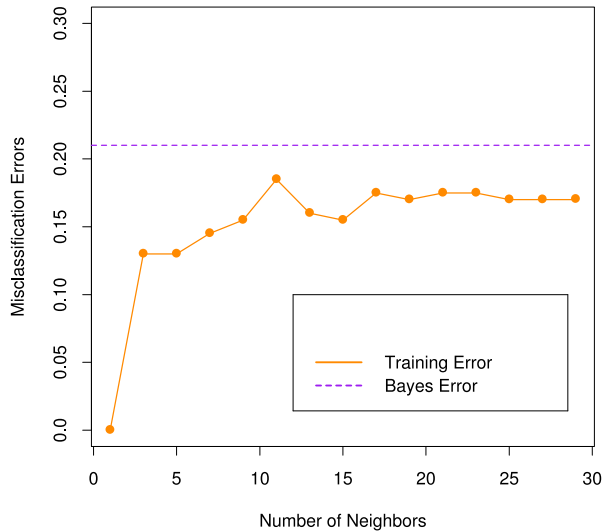
k -Nearest Neighbor Practical Considerations

- How to pick k
- How to pick distance measure (and scaling)
- How to reduce computational costs (not addressed in this class)

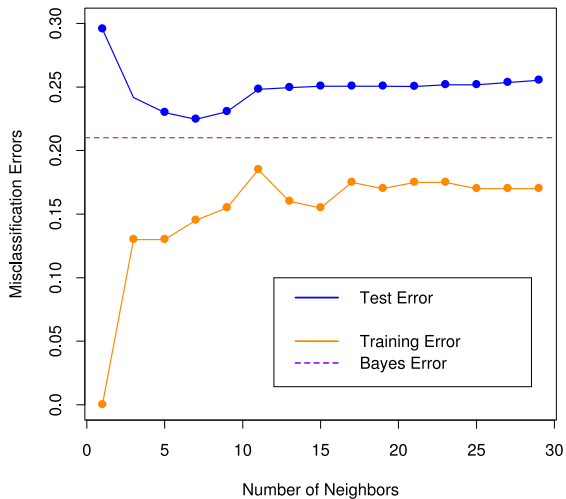
Selection of k



Selection of k

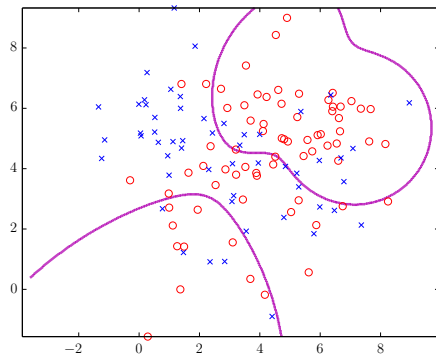


Selection of k

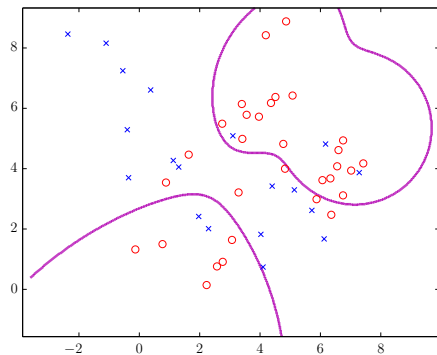


Holdout Validation Example

Training Set

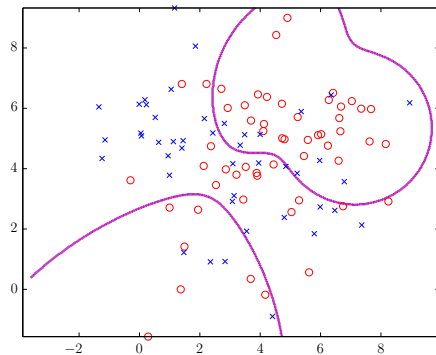


Testing Set

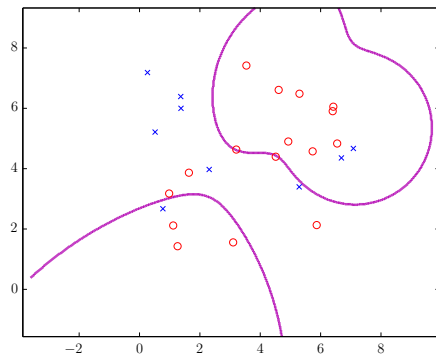


Holdout Validation Example

Training Set

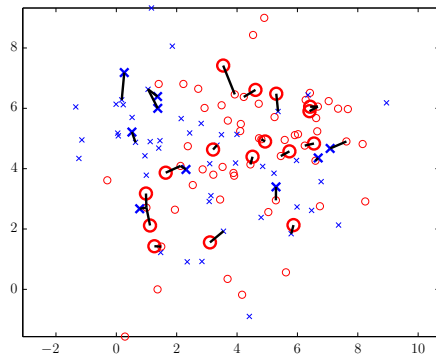


Validation Set

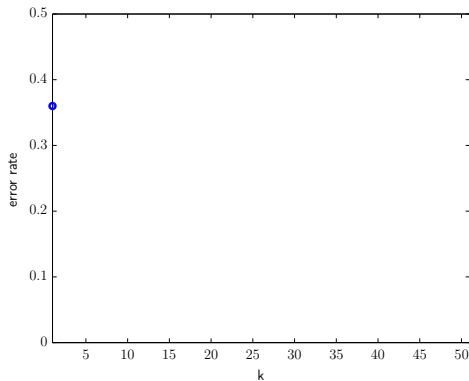


Holdout Validation Example

$k = 1$

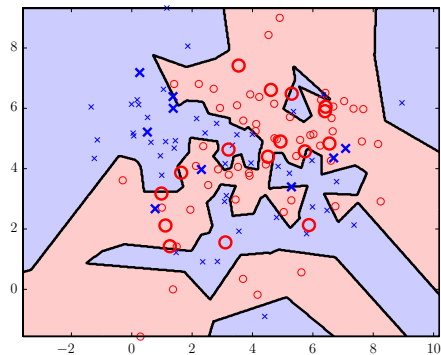


validation error rate

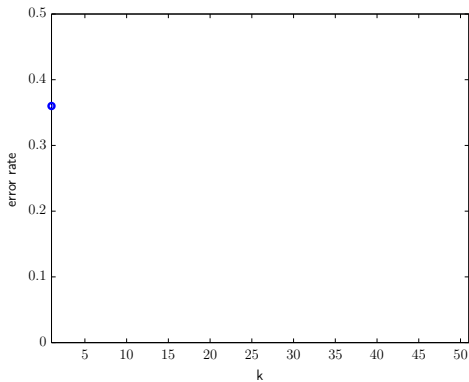


Holdout Validation Example

$k = 1$

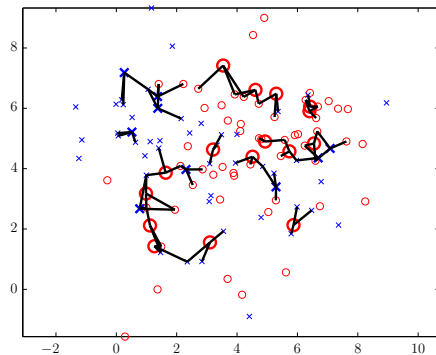


validation error rate

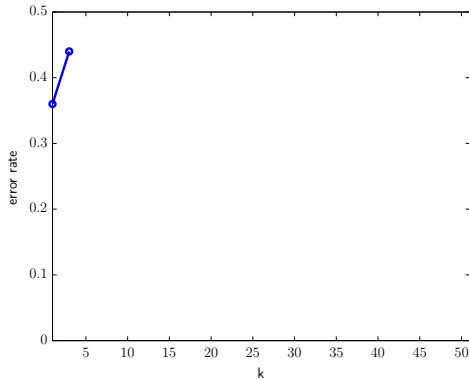


Holdout Validation Example

$k = 1$

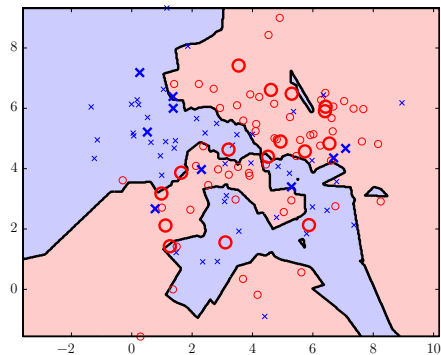


validation error rate

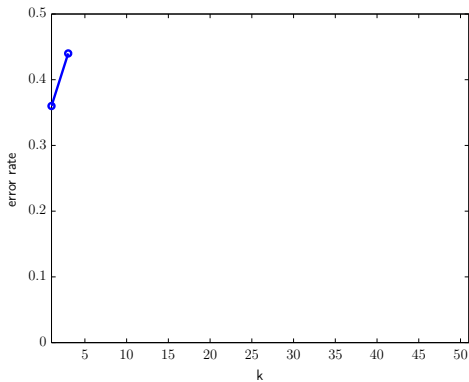


Holdout Validation Example

$k = 3$

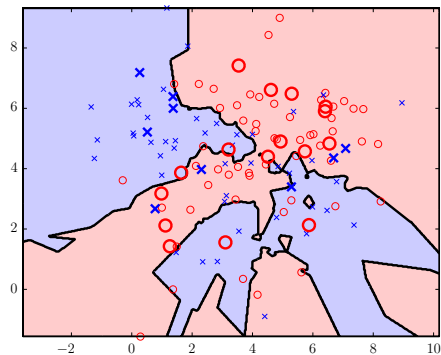


validation error rate

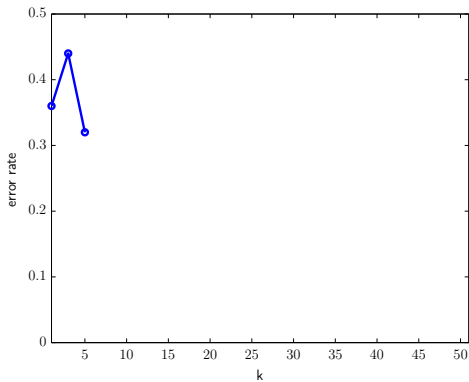


Holdout Validation Example

$k = 5$

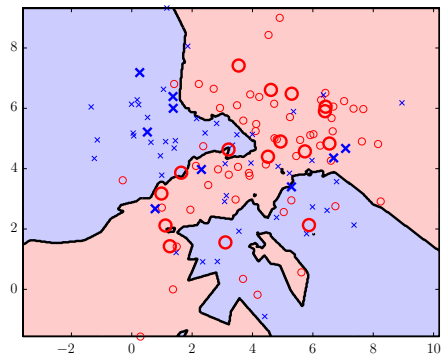


validation error rate

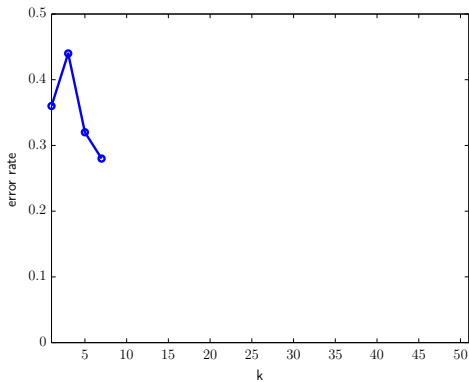


Holdout Validation Example

$k = 7$

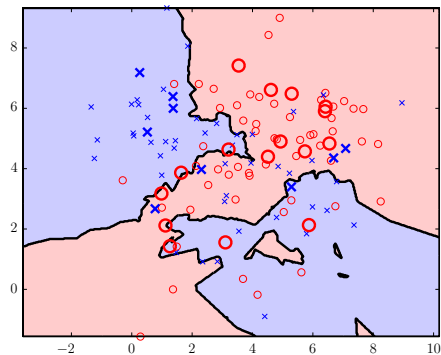


validation error rate

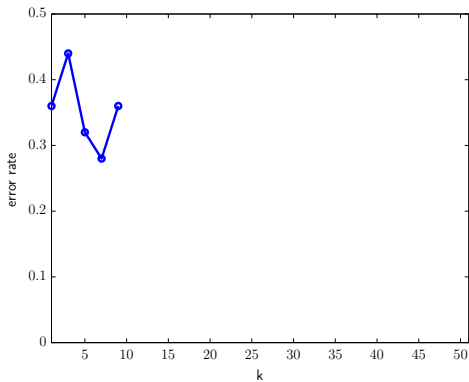


Holdout Validation Example

$k = 9$

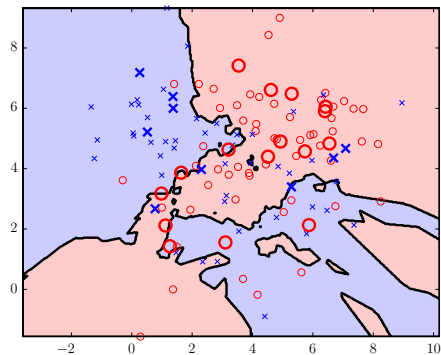


validation error rate

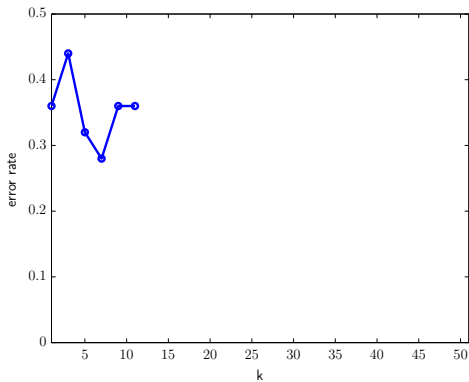


Holdout Validation Example

$k = 11$

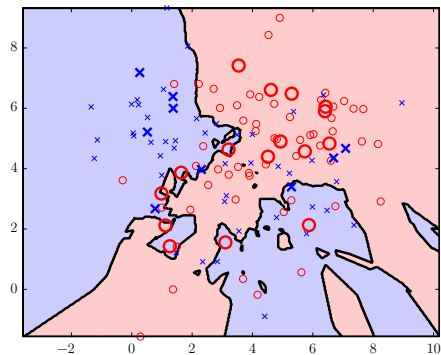


validation error rate

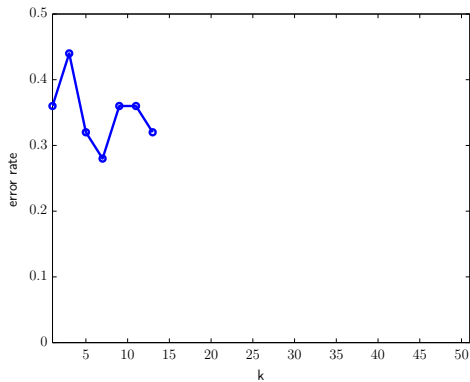


Holdout Validation Example

$k = 13$

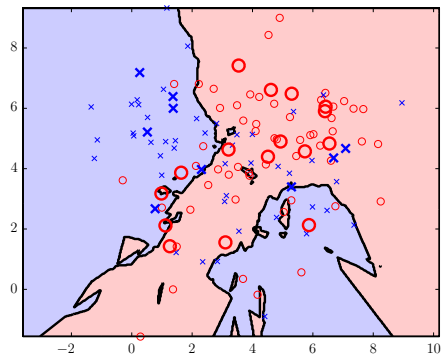


validation error rate

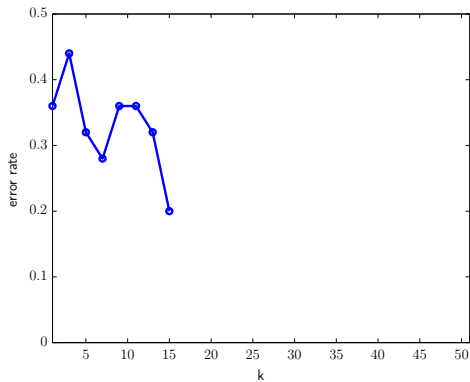


Holdout Validation Example

$k = 15$

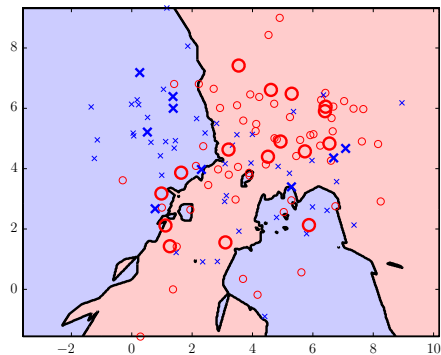


validation error rate

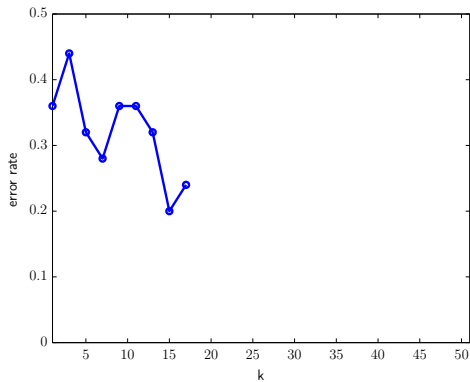


Holdout Validation Example

$k = 17$

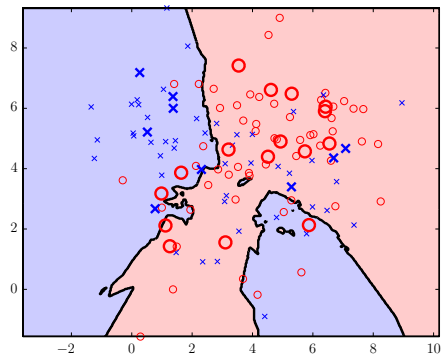


validation error rate

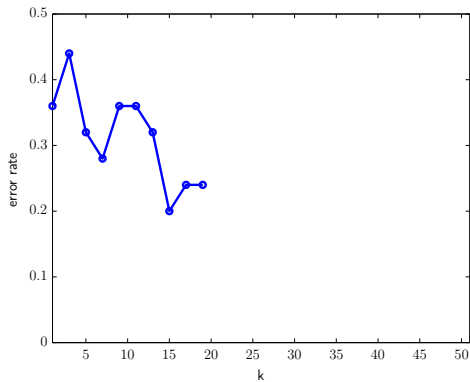


Holdout Validation Example

$k = 19$

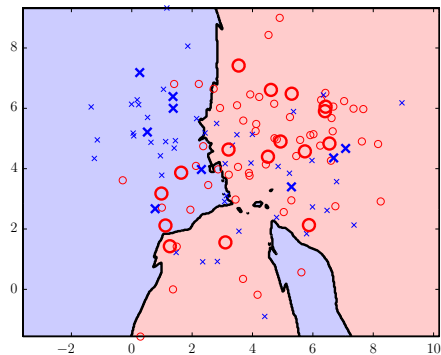


validation error rate

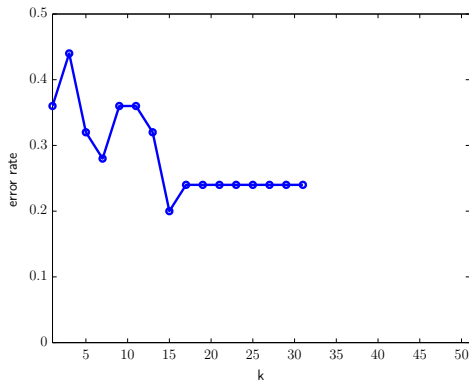


Holdout Validation Example

$k = 31$

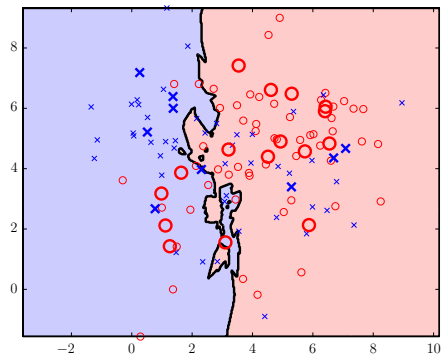


validation error rate

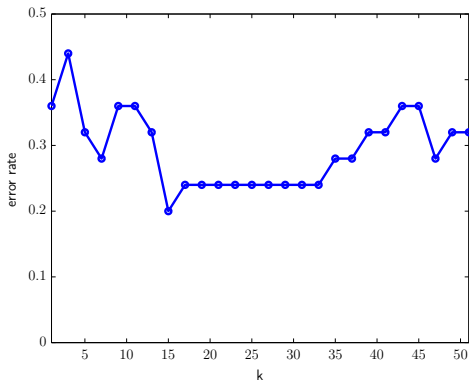


Holdout Validation Example

$k = 51$

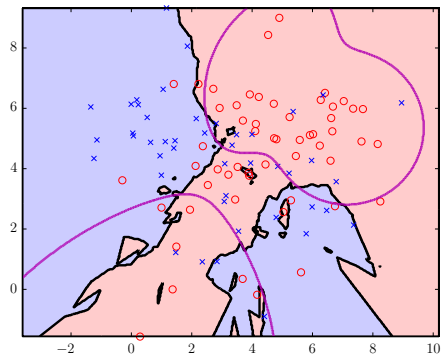


validation error rate

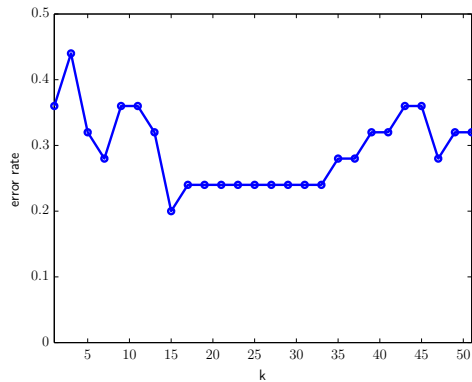


Holdout Validation Example

$k = 15$

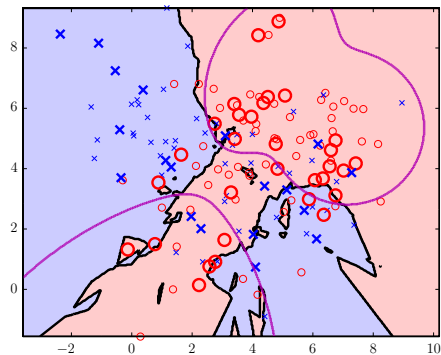


validation error rate

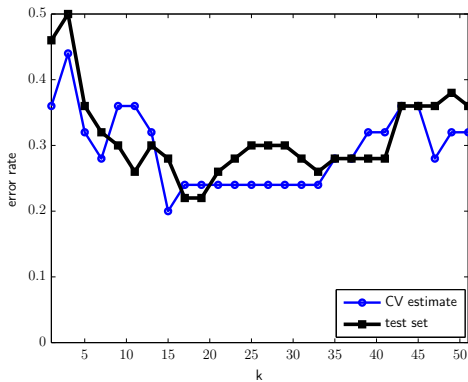


Holdout Validation Example

$k = 15$

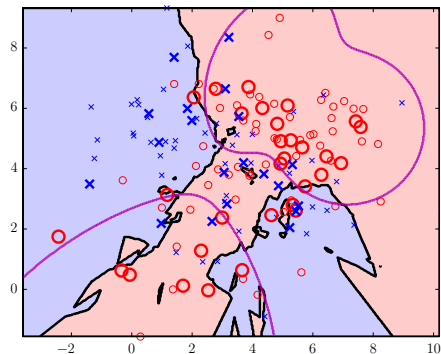


validation error rate

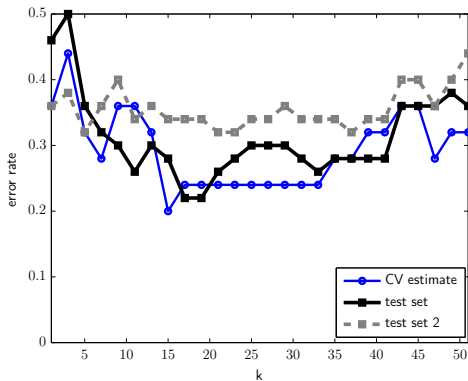


Holdout Validation Example

$k = 15$

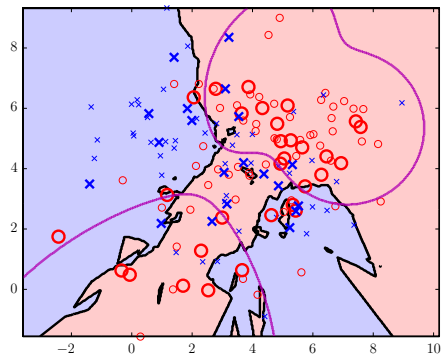


validation error rate

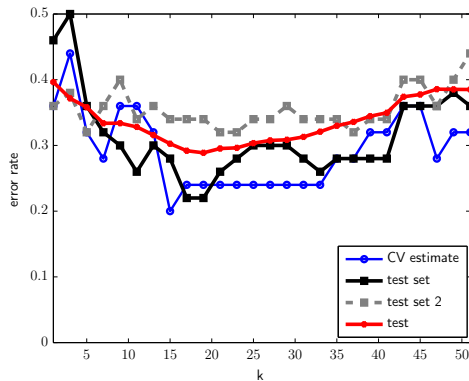


Holdout Validation Example

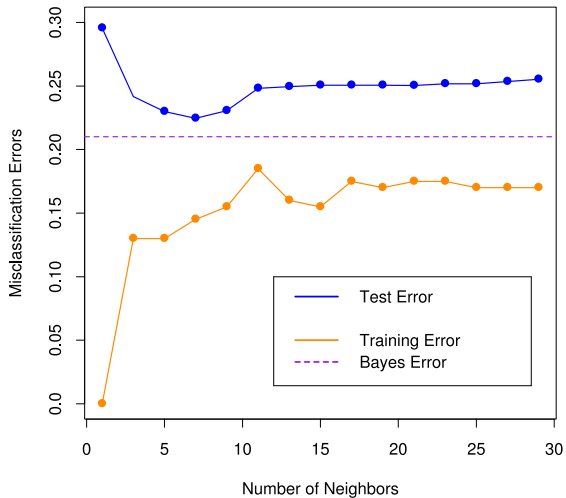
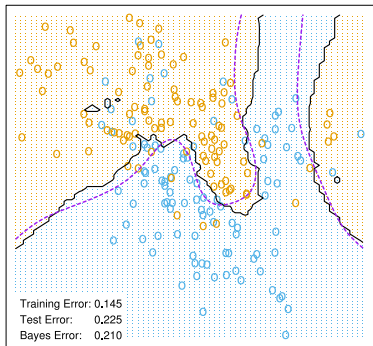
$k = 15$



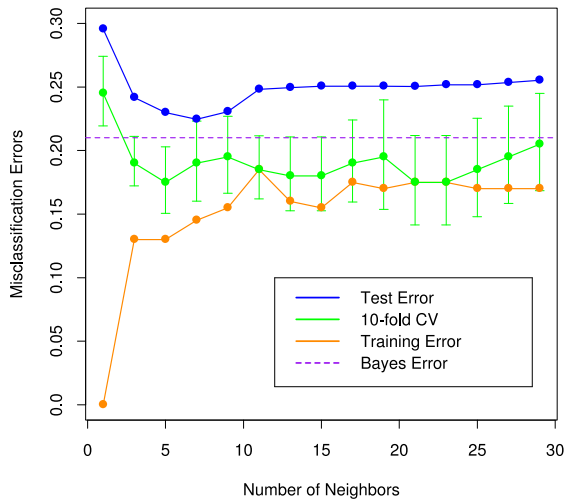
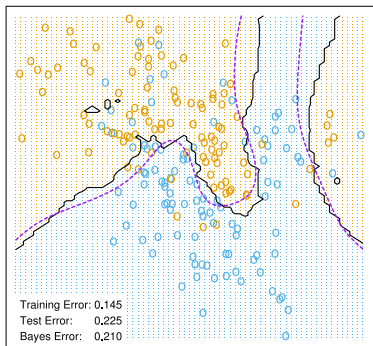
validation error rate



Selection of k



Selection of k



Selection of Distance Metric

- Use cross validation

Selection of Distance Metric

- Use cross validation
- Fine unless you have lots of distance metrics to try (overfitting to the validation set!)
- If distance metric also includes axis scaling, need something else

Example Problem

Problem specification: to classify land use from satellite imagery.

Example Problem

Problem specification: to classify land use from satellite imagery.

- Each pixel corresponds to an area on the Earth.
- Observations are the intensity from four different spectral bands (two visible spectrum, two infrared)
- 7 different classes: red soil, cotton, vegetation stubble, mixture, gray soil, damp gray soil, very damp gray soil

Example Problem

Problem specification: to classify land use from satellite imagery.

- Each pixel corresponds to an area on the Earth.
- Observations are the intensity from four different spectral bands (two visible spectrum, two infrared)
- 7 different classes: red soil, cotton, vegetation stubble, mixture, gray soil, damp gray soil, very damp gray soil

Attributes: the four values from the pixel in question,

Example Problem

Problem specification: to classify land use from satellite imagery.

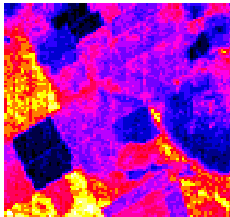
- Each pixel corresponds to an area on the Earth.
- Observations are the intensity from four different spectral bands (two visible spectrum, two infrared)
- 7 different classes: red soil, cotton, vegetation stubble, mixture, gray soil, damp gray soil, very damp gray soil

Attributes: the four values from the pixel in question,
plus the four values from each of the neighboring eight pixels.
(total 36 attributes)

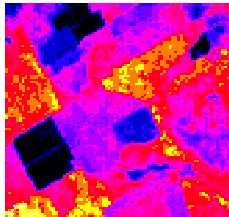
Example Problem

Using $k = 5$, performance is better than any other tried method (error rate of 9.5%).

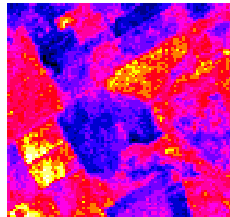
Spectral Band 1



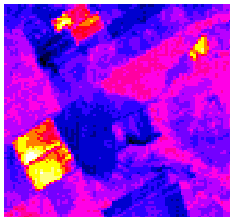
Spectral Band 2



Spectral Band 3



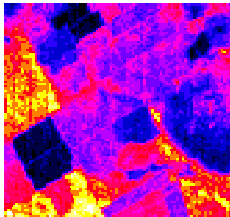
Spectral Band 4



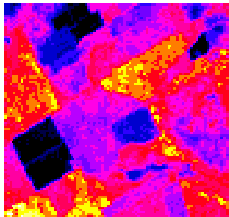
Example Problem

Using $k = 5$, performance is better than any other tried method (error rate of 9.5%).

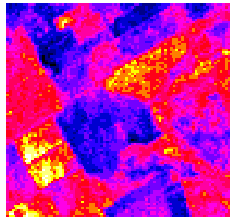
Spectral Band 1



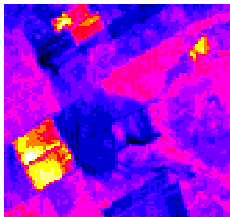
Spectral Band 2



Spectral Band 3



Spectral Band 4



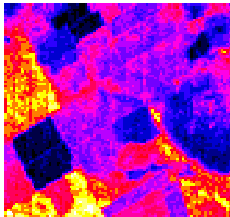
Land Usage



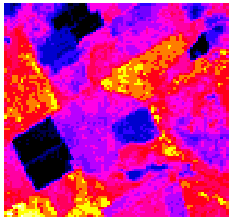
Example Problem

Using $k = 5$, performance is better than any other tried method (error rate of 9.5%).

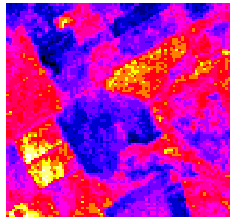
Spectral Band 1



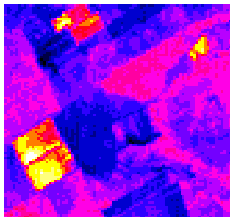
Spectral Band 2



Spectral Band 3



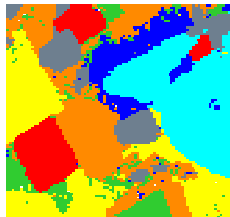
Spectral Band 4



Land Usage



Predicted Land Usage



k -Nearest Neighbor

When is it good?

- Low-dimensional space
- Lots of data
- Lots of computational power
- Highly irregular decision surface